

SQL

(Structured Query Language)

Introduction to SQL

- **SQL** (Structured Query Language) is used to perform operations on the records stored in the database such as updating records, deleting records, creating and modifying tables, views, etc.
- SQL became a standard of the American National Standards Institute (ANSI) in 1986 and of the International Organization for Standardization (ISO) in 1987.
- SQL is just a query language, it is not a database.
- To perform SQL queries, we need to install any database, for example, Oracle, MySQL, MYSQL Server.
- It is designed for managing data in a relational database management system (RDBMS).

What SQL does

- With SQL, we can query our database in several ways, using English-like statements.
- With SQL, a user can access data from a relational database management system.
- It allows the user to define the data in the database and manipulate it when needed.

Tables in SQL

- Table is a collection of data, organized in terms of rows and columns.
- A table can have a specified number of columns but can have any number of rows.
- Each row represents a unique record, and each column represents a field in the record.

SQL Constraints

- Constraints are the rules enforced on the data columns of a table. These are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the database.
- The following constraints are commonly used in SQL:

NOT Null: Ensures that a column cannot have a null value.

Default: Provides a default value for a column when none is specified.

Unique: Ensures that all values in a column are different.

Primary Key: Uniquely identifies each record in a table. Primary keys must contain UNIQUE values, and cannot contain NULL values.

Foreign Key: A foreign key is a field or a column that is used to establish a link between two tables. A foreign key in one table used to point primary key in another table.

DML is abbreviation of Data Manipulation Language. It is used to retrieve, store, modify, delete, insert and update data in database.

SELECT – Retrieves data from a table

INSERT -Inserts data into a table

UPDATE – Updates existing data into a table

DELETE – Deletes all records from a table

DDL is abbreviation of Data Definition Language. It is used to create and modify the structure of database objects in database.

CREATE – Creates objects in the database

ALTER – Alters objects of the database

DROP – Deletes objects of the database

TRUNCATE – Deletes all records from a table and resets table identity to initial value.

DCL is abbreviation of Data Control Language. It is used to create roles, permissions, and referential integrity as well it is used to control access to database by securing it.

GRANT – Gives user's access privileges to database

REVOKE – Withdraws user's access privileges to database given with the GRANT command

TCL is abbreviation of Transactional Control Language. It is used to manage different transactions occurring within a database.

COMMIT – Saves work done in transactions

ROLLBACK – Restores database to original state since the last COMMIT command in transactions

SAVE TRANSACTION – Sets a savepoint within a transaction

SQL SELECT Statement

⑩ SELECT *column1,column2, ...*
FROM *table_name*;

⑩ SELECT * FROM *table_name*;

SQL FROM Statement

SELECT *FROM *table_name*;

SQL WHERE Statement

⑩ SELECT *column1,column2, ...*
FROM *table_name*;
WHERE *condition*;

⑩ SELECT *

⑩ FROM *table_name*;

WHERE *condition*;

SQL GROUP BY Statement

- ⑩ SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
- ⑩ SELECT * FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*

SQL HAVING Statement

- ⑩ SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
HAVING *condition*
- ⑩ SELECT * FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
HAVING *condition*

SQL ORDER BY Statement

- ⑩ SELECT *column_name(s)*
FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
HAVING *condition* ORDER BY *column_name(s)*;
- ⑩ SELECT * FROM *table_name*
WHERE *condition*
GROUP BY *column_name(s)*
HAVING *condition* ORDER BY *column_name(s)*;

SQL JOINS

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

```
⑩ SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

Different Types of SQL JOINS

1.INNER JOIN:-Returns records that have matching values in both tables

```
⑩ SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

2.LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table

```
⑩ SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

3.RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table

```
⑩ SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

4.FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table

```
⑩ SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

SQL Self JOIN

A self JOIN is a regular join, but the table is joined with itself.

```
⑩ SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

The SQL UNION Operator

The UNION operator is used to combine the result-set of two or more SELECT statements.

- ⑩ Each SELECT statement within UNION must have the same number of columns
- ⑩ The columns must also have similar data types
- ⑩ The columns in each SELECT statement must also be in the same order

UNION SYNTAX

- ⑩ SELECT *column_name(s)* FROM *table1*
UNION
SELECT *column_name(s)* FROM *table2*;

UNION ALL SYNTAX

- ⑩ SELECT *column_name(s)* FROM *table1*
UNION ALL
SELECT *column_name(s)* FROM *table2*;

IS NULL SYNTAX

- ⑩ SELECT *column_names*
FROM *table_name*
WHERE *column_name* IS NULL;

IS NOT NULL

- ⑩ SELECT *column_names*
FROM *table_name*
WHERE *column_name* IS NOT NULL;

LIKE SYNTAX

- ⑩ SELECT *column1, column2, ...*
FROM *table_name*
WHERE *column* LIKE *pattern*;

LIKE Operator

Description

WHERE CustomerName LIKE 'a%' Finds any values that start with "a"

WHERE CustomerName LIKE '%a' Finds any values that end with "a"

WHERE CustomerName LIKE '%or%' Finds any values that have "or" in any position

WHERE CustomerName LIKE '_r%' Finds any values that have "r" in the second position

WHERE CustomerName LIKE 'a_%' Finds any values that start with "a" and are at least 2 characters in length

WHERE CustomerName LIKE 'a__%' Finds any values that start with "a" and are at least 3 characters in length

WHERE ContactName LIKE 'a%o' Finds any values that start with "a" and ends with "o"

SQL Create Constraints

Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement.

```
⑩ CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    ....  
)
```

The following constraints are commonly used in SQL:

NOT NULL- Ensures that a column cannot have a NULL value

```
⑩ CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255) NOT NULL,  
    Age int  
);
```

⑩ ALTER TABLE Persons
MODIFY Age int NOT NULL;

UNIQUE - Ensures that all values in a column are different

⑩ CREATE TABLE Persons (
ID int NOT NULL UNIQUE,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Age int
);

⑩ ALTER TABLE Persons
ADD CONSTRAINT UC_Person UNIQUE (ID,LastName);

PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table

⑩ CREATE TABLE Persons (
ID int NOT NULL PRIMARY KEY,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Age int
);

⑩ ALTER TABLE Persons
ADD CONSTRAINT PK_Person PRIMARY KEY (ID,LastName);

FOREIGN KEY - Uniquely identifies a row/record in another table

⑩ CREATE TABLE Orders (
OrderID int NOT NULL PRIMARY KEY,
OrderNumber int NOT NULL,
PersonID int FOREIGN KEY REFERENCES Persons(PersonID)
);

⑩ ALTER TABLE Orders
ADD CONSTRAINT FK_PersonOrder
FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);

CHECK- Ensures that all values in a column satisfies a specific condition

```
⑩ CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int CHECK (Age>=18)  
);  
  
⑩ ALTER TABLE Persons  
ADD CHECK (Age>=18);
```

DEFAULT - Sets a default value for a column when no value is specified

```
⑩ CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Sandnes'  
);  
  
⑩ ALTER TABLE Persons  
ADD CONSTRAINT df_City  
DEFAULT 'Sandnes' FOR City;
```

INDEX - Used to create and retrieve data from the database very quickly

Creates an index on a table. Duplicate values are allowed:

```
⑩ CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

Creates a unique index on a table. Duplicate values are not allowed:

```
⑩  
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

AUTO INCREMENT Field

Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table.

Often this is the primary key field that we would like to be created automatically every time a new record is inserted.

```
⑩ CREATE TABLE Persons (  
    Personid int NOT NULL AUTO_INCREMENT,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    PRIMARY KEY (Personid)  
);
```

Subquery

A subquery can define as one query embedded in another query.

A normal SQL query divided into two parts one is inner query and another one is the outer query the inner query.

The inner query is called subquery which is placed in the round parentheses after the WHERE clause, HAVING clause or FROM clause.

The outer query is also called the main query which is dependent on the inner query.

The outer query gets executed after the subquery generates a result that is used by the outer query to produce the output.

Expression(Comparison) operator like <,>,,≤,=,<=,≥in the SQL is used in the subquery to compare both queries and give output.

The subquery as to be placed on the right side of the expression operator.

Syntax:

```
⑩ Select * from table_name where Column_name(s) →outer query
```

⑩ comparison operator

⑩ (Select Column_Name(s) from table_name); → *inner query*

Correlated subquery:

Correlated Subquery is a type of subquery.

Correlated Subquery is different from the normal subquery in terms of execution.

In this query, the correlated subquery is evaluated once for each row of the outer query.

The Correlated subquery is slower compared to a normal subquery.

It depends on the outer query.

Syntax:

- ⑩ SELECT column_name(s)
- ⑩ FROM table1_name mainq
- ⑩ WHERE column_name(s) operator
- ⑩ (SELECT column_name(s) FROM table2_name
- ⑩ WHERE expr1 =mainq.expr2);

ROW_NUMBER()

Assigns sequential numbers to each partition in a result set (an unpartitioned result set simply has a single partition), based upon the order of the results as specified in the ORDER BY clause.

If you look carefully, you'll see that the values in column ROWNO are based upon a simple sort of Branches.

⑩ ROW_NUMBER() OVER ([<partition_by_clause>] <order_by_clause>)

RANK()

This function does much the same thing as ROW_NUMBER(), only it acknowledges ties in the columns specified in the ORDER BY clause, and assigns them the same rank.

Where a tie occurs, the numbers that would otherwise have been "used up" are skipped,

and numbering resumes at the next available number. As you can see, RANK() leaves a gap whenever there is a tie.

⑩ **RANK() OVER ([<partition_by_clause>] <order_by_clause>)**

DENSE_RANK()

The DENSE_RANK() is a Window function that assigns a rank to each row within a partition of a result set. Unlike the Rank () function, the DENSE_RANK() function returns consecutive rank values. Rows in each partition receive the same ranks if they have the same values.

⑩ **DENSE_RANK () OVER ([<partition_by_clause>] <order_by_clause>)**

The syntax of the DENSE_RANK()

```
DENSE_RANK() OVER (  
    [PARTITION BY partition_expression, ... ]  
    ORDER BY sort_expression [ASC | DESC], ...  
)
```

Functions in SQL:

CHARINDEX:

The CHARINDEX() function searches for a substring in a string, and returns the position.

If the substring is not found, this function returns 0.

Note: This function performs a case-insensitive search.

Syntax

⑩ **CHARINDEX(substring, string, start)**

substring Required. The substring to search for

string Required. The string to be searched

start Optional. The position where the search will start (if you do not want to start at

the beginning of string). The first position in string is 1

SUBSTRING()

The SUBSTRING() function extracts some characters from a string.

Syntax:

⑩ SUBSTRING(string, start, length)

string Required. The string to extract from

start Required. The start position. The first position in string is 1

length Required. The number of characters to extract. Must be a positive number

LEN:

The LEN() function returns the length of a string.

Note: Trailing spaces at the end of the string is not included when calculating the length.

However, leading spaces at the start of the string is included when calculating the length.

Tip:Also look at the DATALENGTH()function.

Syntax:

LEN(string)

string Required. The string to return the length for. If string is NULL, it returns NULL

TRIM:

The TRIM() function removes the space character OR other specified characters from the start or end of a string.

By default, the TRIM() function removes leading and trailing spaces from a string.

Note:Also look at the LTRIM() and RTRIM() functions.

Syntax:

⑩ TRIM([charactersFROM]string)

LTRIM:

The LTRIM() function removes leading spaces from a string.

Syntax

⑩ LTRIM(string)

string Required. The string to remove leading spaces from

RTRIM:

The RTRIM() function removes trailing spaces from a string.

Syntax:

⑩ RTRIM(string)

REPLACE:

The REPLACE() function replaces all occurrences of a substring within a string, with a new substring.

Note: The search is case-insensitive.

Syntax:

⑩ REPLACE(string, old_string, new_string)

Replicate:

The REPLICATE() function repeats a string a specified number of times.

Syntax:

⑩ REPLICATE(string, integer)

string Required. The string to repeat

integer Required. The number of times to repeat the string

Patindex:

The PATINDEX() function returns the position of a pattern in a string.

If the pattern is not found, this function returns 0.

Note: The search is case-insensitive and the first position in string is 1.

Syntax:

PATINDEX(%pattern%,string)

Parameter	Description
-----------	-------------

%patter%	Required. The pattern to find. It MUST be surrounded by %. Other wildcards can be used in pattern, such as:
----------	---

- ⑩ % - Match any string of any length (including 0 length)
 - ⑩ _ - Match one single character
 - ⑩ [] - Match any characters in the brackets, e.g. [xyz]
 - ⑩ [^] - Match any character not in the brackets, e.g. [^xyz]
- string Required. The string to be searched

Example:

Return the position of a pattern in a string:

```
SELECT PATINDEX('%schools%', 'W3Schools.com');
```

Output: 3

Example:

Return the position of a pattern in a string:

```
SELECT PATINDEX('%[ol]%', 'W3Schools.com');
```

Output:6

Example:

Return the position of a pattern in a string:

```
SELECT PATINDEX('%[z]%', 'W3Schools.com');
```

Output:0

SQL CAST Function

The SQL CAST is a SQL Conversions Function used to convert an expression from one data type to another. If the SQL Server CAST function is unable to convert an expression into the desired data type, this function will return Error.

The syntax of SQL Server CAST Function is.

```
CAST (Expression AS Data_Type [(Length)])
```

Cast in SQL example

```
SELECT CAST (123 AS VARCHAR(50)) AS [result_name]FROM [Source]
```

- ⑩ Expression:Valid expression that you want to convert into the desired data type in SQL Server.
- ⑩ Data_Type:Specify the Data Type to which you want to convert an expression
- ⑩ Length:This is an optional parameter of integer type. You can use this parameter to define the length of the target data type. By default, it is 30.

Example:

```
-- SQL Server CAST FUNCTION Example DECLARE @str AS VARCHAR(50)SET @str = '12121'SELECT CAST(@str AS INT) AS Result;
```

```
-- Direct Inputs
```

```
SELECT CAST('1234' AS DECIMAL(10, 2)) AS Result;
```

```
SELECT CAST(123.456 AS INT) AS Result;
```

```
SELECT CAST('07/03/2017' AS DATETIME) AS Result;
```

```
SELECT CAST('07/03/2017' AS DATETIME2) AS Result
SELECT CAST(GETDATE() AS VARCHAR(50)) AS Result;
```

Output:

Result:12121

Result :1234.00

Result :123

Result: 2017-07-03 00:00:00:000

Result: 2017-07-03 00:00:00:00000000

Result: Mar 7 2017 4:07PM

Convert:

The CONVERT() function converts a value (of any type) into a specified datatype.

Syntax:

CONVERT(data_type(length), expression, style)

Parameter Values

Value	Description
data_type	Required. The datatype to convert expression to. Can be one of the following: bigint, int, smallint, tinyint, bit, decimal, numeric, money, smallmoney, float, real, datetime, smalldatetime, char, varchar, text, nchar, nvarchar, ntext, binary, varbinary, or image
(length)	Optional. The length of the resulting data type (for char, varchar, nchar, nvarchar, binary and varbinary)
expression	Required. The value to convert to another data type
style	Optional. The format used to convert between data types, such as a date or string format. Can be one of the following values:

Import and Export wizard in SQL Server:

We have another option in SQL Server Management Studio (SSMS) to import the data from flat file to SQL table and the steps as follows.

Step 1: Right click on the database where we need to import and choose Tasks from the list and click Import Data option.

Step 2: In Choose a data source window, choose Flat File Source from the data source list and select the file path using Browse button.

Step 3: Type double quotes (") on Text qualifier text box and preview the data choosing columns in left panel then click Next.

Step 4: In Choose a Destination window, choose SQL Server Native Client 11.0 from the list and mention the SQL server and credential details then click Next.

Step 5: In Select Source Tables and Views window, Type the destination table or choose existing table.

Step 6: Click Edit Mappings button and verify the table columns, data types and length.

Step 7: Click Next and Finish button if all the columns mapped appropriately.

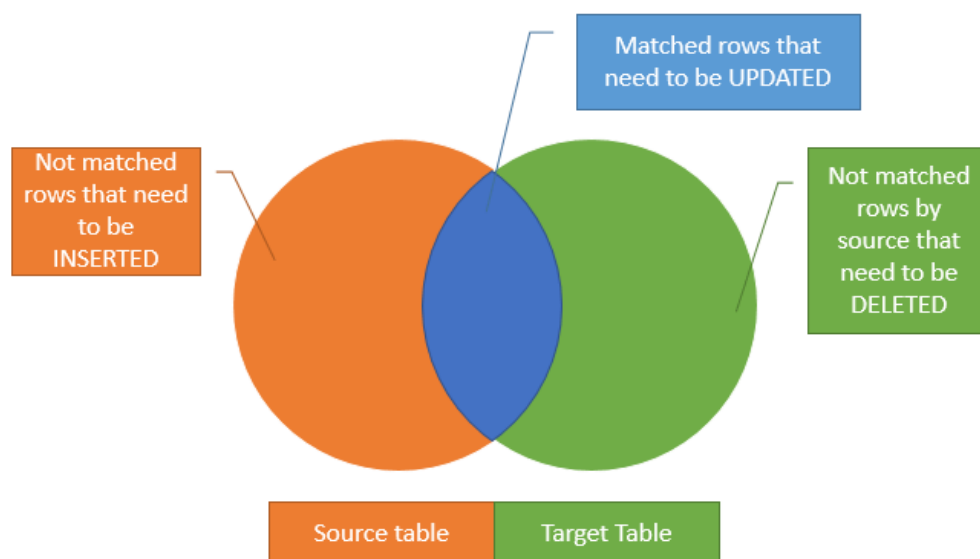
Merge:

Suppose, you have two table called source and target tables, and you need to update the target table based on the values matched from the source table. There are three cases:

- 1.The source table has some rows that do not exist in the target table. In this case, you need to insert rows that are in the source table into the target table.
- 2.The target table has some rows that do not exist in the source table. In this case, you need to delete rows from the target table.
- 3.The source table has some rows with the same keys as the rows in the target table.

However, these rows have different values in the non-key columns. In this case, you need to update the rows in the target table with the values coming from the source table.

The following picture illustrates the source and target tables with the corresponding actions: insert, update, and delete:



If you use the INSERT, UPDATE, and DELETE statement individually, you have to construct three separate statements to update the data to the target table with the matching rows from the source table.

However, SQL Server provides the MERGE statement that allows you to perform three actions at the same time. The following shows the syntax of the MERGE statement:

```
MERGE target_table USING source_table ON merge_condition WHEN MATCHED THEN  
update_statement WHEN NOT MATCHED THEN insert_statement WHEN NOT MATCHED  
BY SOURCE THEN DELETE;
```

Intersect:

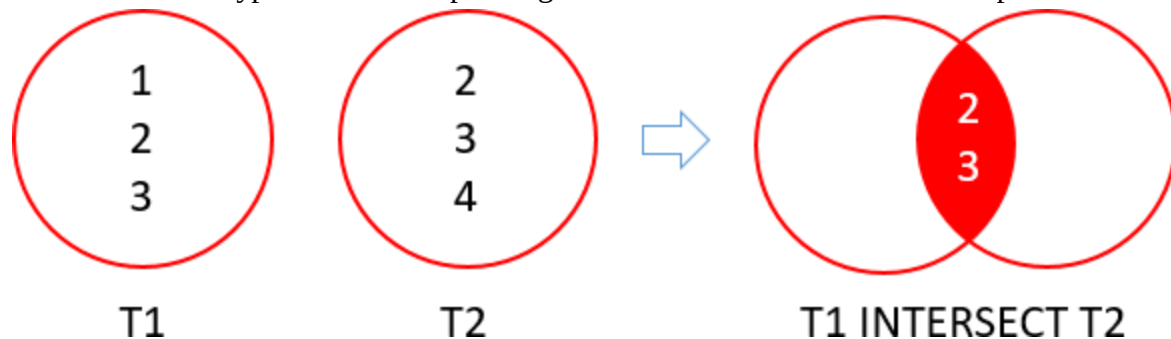
The SQL Server INTERSECT combines result sets of two or more queries and returns distinct rows that are output by both queries.

The following illustrates the syntax of the SQL Server INTERSECT:

```
query_1 INTERSECT query_2
```

Similar to the UNION operator, the queries in the syntax above must conform to the following rules:

- ⑩ Both queries must have the same number and order of columns.
- ⑩ The data type of the corresponding columns must be the same or compatible.



The following picture illustrates the INTERSECT operation:

In this illustration, we had two result sets T1 and T2:

- ⑩ T1 result set includes 1, 2, and 3.
- ⑩ T2 result set includes 2, 3, and 4.

The intersection of T1 and T2 result sets returns the distinct rows which are 2 and 3.

Except:

The SQL Server EXCEPT compares the result sets of two queries and returns the distinct rows from the first query that are not output by the second query. In other words, the EXCEPT subtracts the result set of a query from another.

The following shows the syntax of the SQL Server EXCEPT:

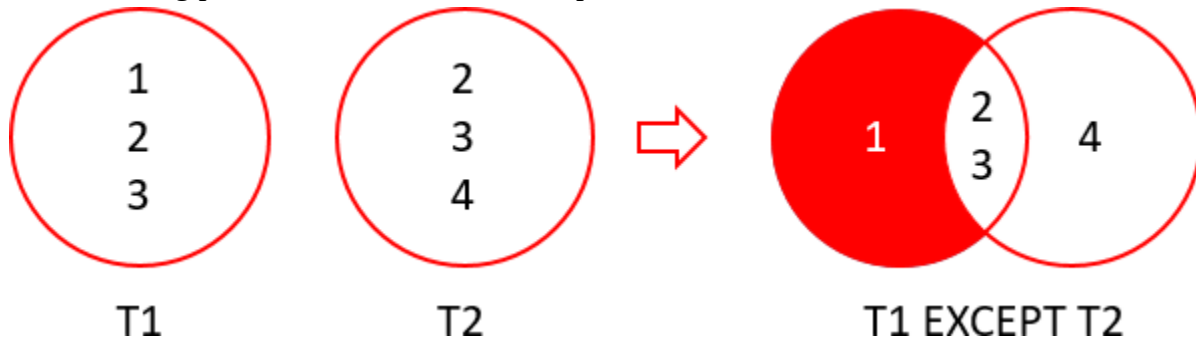
```
query_1 EXCEPT query_2
```

The following are the rules for combining the result sets of two queries in the above syntax:

- ⑩ The number and order of columns must be the same in both queries.

- ⑩ The data types of the corresponding columns must be the same or compatible.

The following picture shows the EXCEPT operation of the two result sets T1 and T2:



In this illustration:

- ⑩ T1 result set includes 1, 2, and 3.
- ⑩ T2 result set includes 2, 3, and 4.

The except of the T1 and T2 returns 1 which is the distinct row from the T1 result set that does not appear in the T2 result set.

CASE:

The CASE statement goes through conditions and returns a value when the first condition is met (like an IF-THEN-ELSE statement). So, once a condition is true, it will stop reading and return the result. If no conditions are true, it returns the value in the ELSE clause.

If there is no ELSE part and no conditions are true, it returns NULL.

Syntax

CASE Syntax:

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

WHEN conditionN THEN resultN

ELSE result

END;

Example:

SELECT CustomerName, City, Country

FROM Customers

ORDER BY

(CASE

WHEN City IS NULL THEN Country

ELSE City
END);

SQL CREATE VIEW Statement

In SQL, a view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

Syntax:

```
⑩ CREATE VIEW view_name AS  
  SELECT column1, column2, ...  
  FROM table_name  
  WHERE condition;
```

SQL CREATE INDEX Statement

The CREATE INDEX statement is used to create indexes in tables.

Indexes are used to retrieve data from the database more quickly than otherwise. The users cannot see the indexes, they are just used to speed up searches/queries.

Syntax:

```
⑩ CREATE INDEX index_name (Duplicate values are allowed )  
  ON table_name (column1, column2, ...);  
  
⑩ CREATE UNIQUE INDEX index_name (Duplicate values are not allowed )  
  ON table_name (column1, column2, ...);
```

SQL Server CREATE TRIGGER

The CREATE TRIGGER statement allows you to create a new trigger that is fired automatically whenever an event such as INSERT, DELETE, or UPDATE occurs against a table.

Syntax:

```
CREATE TRIGGER [schema_name.]trigger_name
```

ON table_name

AFTER {[INSERT],[UPDATE],[DELETE]}

[NOT FOR REPLICATION] AS {sql_statements}

IIF IN SQL

The IIF() function returns a value if a condition is TRUE, or another value if a condition is FALSE.

Syntax:

⑩ IIF(*condition, value_if_true, value_if_false*)

Documentation by

cherukumalli sai sucharith

Associate trainee

SQL TRAINING BY

venkateswarlu upplapati

ETL TEAM LEAD